Implementazione di BB84 in Qiskit

Marta Finazzi 84401A

Febbraio 2026

Indice

1 Il Protocollo BB84 **2**
1.1 I qubit 2
1.2 Terminologia e funzionamento del protocollo 2
1.2.1 Funzionamento senza intercettazione 3
1.2.2 Funzionamento con intercettazione 4
1.3 Sicurezza del protocollo 5

2 Implementazione in Qiskit **6**
2.1 Strumenti Software: Qiskit e Aer 6
2.2 Gate Quantistici e Operatori 6
2.3 Analisi del codice 7

3 Analisi dei Risultati **9**
3.1 Esempio d'esecuzione senza intercettazione 9
3.2 Esempio d'esecuzione con intercettazione 10
3.2.1 Eve intercettata 10
3.2.2 Eve non intercettata 11
3.3 Analisi Statistica e Convergenza 12
3.3.1 Dati Sperimentali 12
3.3.2 Interpretazione dei Risultati 12

Conclusioni **14**

Capitolo 1

Il Protocollo BB84

Il protocollo BB84 è un sistema di Quantum Key Distribution (QKD) proposto da Charles Bennett e Gilles Brassard nel 1984. Esso utilizza la polarizzazione dei fotoni (particelle di luce associate ai qubit) per codificare i bit di informazione. A differenza dei protocolli tradizionali, la cui complessità dipende da problemi matematici difficili (come RSA e la fattorizzazione di grandi numeri), BB84 sfrutta i principi della meccanica quantistica per garantirne la sicurezza.

Le tecnologie stanno ormai avanzando velocemente, e algoritmi quantistici, come quello di Shor, riescono a violare i crittosistemi tradizionali. BB84 rappresenta la risposta Post-Quantum e garantisce sicurezza e la capacità di rilevare immediatamente la presenza di un eventuale intercettatore.

1.1 I qubit

Il qubit è l'unità fondamentale dell'informazione quantistica e può essere visto come il corrispettivo di un bit nell'informatica classica, con l'eccezione che se un bit può valere 0 o 1, un qubit esiste come sovrapposizione di tutti gli stati disponibili. Questo fenomeno viene espresso tramite la proprietà di *superposition*.

Questo significa che finché non viene misurato, un qubit non ha un valore preciso. Una volta effettuata la misurazione, il valore rimane fisso.

1.2 Terminologia e funzionamento del protocollo

La codifica dei bit da inviare avviene tramite la polarizzazione di fotoni sfruttando due basi: la base rettilinea e la base diagonale. Affinché la comunicazione sia coerente, mittente (Alice) e destinatario (Bob) devono concordare preventivamente una convenzione per l'interpretazione dei simboli. In questa implementazione, utilizzeremo la seguente mappatura:

- **Base Z (Rettilinea):** $|0\rangle \rightarrow$ bit $0, |1\rangle \rightarrow$ bit 1.
- **Base X (Diagonale):** $|+\rangle \rightarrow$ bit $0, |-\rangle \rightarrow$ bit 1.

Efficienza e Lunghezza della Chiave È fondamentale notare che la lunghezza della chiave finale prodotta sarà sempre inferiore o, al massimo, uguale alla stringa iniziale di qubit inviati. Poiché Alice e Bob scelgono le basi in modo indipendente e casuale, la probabilità che queste coincidano è del 50%. Di conseguenza, il processo di *sifting* comporta mediamente lo scarto della metà dei bit trasmessi, rendendo la lunghezza della chiave finale circa pari al 50% della stringa di partenza.

Per valutare l'affidabilità della comunicazione e l'eventuale presenza di un intercettatore, vengono analizzati due parametri statistici fondamentali derivati dal confronto dei bit dopo il *sifting*: la **Fidelity** e il **QBER**.

La **Fidelity** rappresenta la percentuale di bit della chiave finale che coincidono tra Alice e Bob; in una simulazione ideale priva di disturbi, questo valore è pari al 100%. Al contrario, la **Loss** (tecnicamente indicata come **QBER**, *Quantum Bit Error Rate*) misura il tasso di errore riscontrato. In presenza di un attacco **Intercept-Resend**, la Fidelity scende teoricamente verso il 75%, mentre la Loss sale al 25%. Questa discrepanza non è dovuta a limiti tecnici, ma è la prova fisica del collasso della funzione d'onda causato dalla misura di Eve: poiché l'intercettatore non conosce le basi di Alice, la sua interferenza introduce errori casuali che rendono la presenza di un intruso matematicamente rilevabile.

1.2.1 Funzionamento senza intercettazione

1. Alice genera una stringa di bit casuali.
2. Alice sceglie casualmente delle basi con cui codificare i bit.
3. Alice invia i bit codificati a Bob.
4. Bob sceglie casualmente delle basi con cui decodificare i bit ricevuti. Se Bob usa la stessa base di Alice, otterrà il bit corretto con probabilità 100%. Se usa la base sbagliata, otterrà 1 o 0 con probabilità 50%.
5. Alice e Bob inviano a vicenda le basi utilizzate sul canale pubblico.
6. Alice e Bob salvano i bit decifrati con la stessa base (Sifting). Questi bit formano la chiave.

Esempio di esecuzione:

Bit Alice	0	0	1	1	0	1	0
Base Alice	X	Z	X	X	X	Z	Z
Base Bob	X	X	X	Z	Z	Z	Z
Bit Bob	0	0/1	1	0/1	0/1	1	0
Esito	**Ok**	-	**Ok**	-	-	**Ok**	**Ok**

Chiave finale: 0110

1.2.2 Funzionamento con intercettazione

1. Alice genera una stringa di bit casuali.
2. Alice sceglie casualmente delle basi con cui codificare i bit.
3. Alice invia i bit codificati a Bob.
4. **Eve intercetta** i bit e li misura con una base casuale, disturbando lo stato originale (attacco **Intercept-Resend**).
5. Eve invia i qubit (disturbati) a Bob.
6. Bob decodifica con basi casuali.
7. Alice e Bob confrontano le basi e verificano la correttezza di un campione della chiave. Potrebbe capitare che Alice e Bob abbiano usato la stessa base ma ottenuto bit diversi per via della presenza di Eve. Se il tasso d'errore supera una certa soglia, solitamente prefissata all'11%, rilevano la presenza di un intercettatore e devono eseguire nuovamente il protocollo. Se il tasso di errore ottenuto è inferiore alla soglia prefissata, si sceglie come chiave quella di Alice.

Esempio di esecuzione (con intercettazione di Eve):

Bit Alice	0	0	1	1	0	1	0
Base Alice	X	Z	X	X	X	Z	Z
Base Eve	Z	Z	X	Z	X	X	Z
Bit Eve	0/1	0	1	0/1	0/1	0/1	0
Base Bob	X	X	X	Z	Z	Z	Z
Bit Bob	0/1	0/1	1	0/1	0/1	0/1	0
Esito	**Errore?**	-	**Ok**	-	-	**Errore?**	**Ok**

Nel momento in cui Alice e Bob si comunicano le basi, scelgono anche di verificare se i bit relativi alle stesse basi sono effettivamente uguali. Considerando la tabella sopra, Alice ha usato le basi X-Z-X-X-X-Z-Z e Bob X-X-X-Z-Z-Z-Z; di

conseguenza, la chiave dovrebbe essere formata dal primo, terzo, sesto e settimo bit.

A queste posizioni, Alice possiede i bit 0-1-1-0. Bob, a causa dell'interferenza, ha ottenuto risultati incerti nelle posizioni in cui Eve ha usato la base errata. Supponiamo che Bob abbia ottenuto la sequenza 1-1-1-0. Alice e Bob scelgono di verificare alcuni bit della chiave (ad esempio il primo e il secondo) e rilevano che, nonostante entrambi abbiano usato la base X per il primo bit, hanno ottenuto due risultati diversi (Alice 0, Bob 1). Il secondo bit invece risulta essere corretto.

Alice e Bob calcolano quindi il tasso di errore QBER)come il rapporto tra i bit non coincidenti e il numero totale di bit verificati:

$$QBER = \frac{\text{Bit Errati}}{\text{Bit Analizzati}} = \frac{1}{2} = 0.5 \rightarrow 50\% > 11\%$$

Eve è stata quindi scoperta ed è necessario effettuare nuovamente il protocollo.

1.3 Sicurezza del protocollo

La sicurezza del protocollo è dimostrabile matematicamente e si basa sul Teorema di Non-Clonazione, che afferma che è impossibile creare una copia identica di uno stato quantistico sconosciuto.

Mentre nel mondo classico un segnale può essere intercettato e duplicato senza alterarne l'originale, nel dominio quantistico l'atto di "copiare" richiede necessariamente una misurazione che porta al cambiamento di stato del qubit. Di conseguenza, qualsiasi tentativo da parte di un intercettatore (Eve) di duplicare i fotoni inviati lascerà tracce indelebili sotto forma di errori statistici, rendendo impossibile eseguire dello spionaggio passivo tramite un attacco **Intercept-Resend**.

Infatti, nel momento in cui un ascoltatore intercetta la comunicazione e tenta di misurare i fotoni, ha il 50% di probabilità di scegliere la base corretta. Se sbaglia, il destinatario, anche se usa la stessa base del mittente, otterrà un risultato errato nel 50% dei casi. Di conseguenza l'errore teorico introdotto da un intercettatore è pari a:

$$0.5\,(\text{prob. errore base}) \times 0.5\,(\text{prob. errore misura}) = 0.25 \quad (25\%)$$

Tuttavia, nella pratica implementativa, è necessario stabilire una soglia di tolleranza nota come *soglia di sicurezza*, solitamente fissata all'11%. Questo margine è fondamentale per distinguere il disturbo causato da un attaccante dal rumore di fondo intrinseco dei dispositivi hardware.

Va inoltre considerato che la rilevazione di Eve dipende fortemente dalla **significatività statistica** del campione: su un numero ridotto di qubit, le fluttuazioni casuali potrebbero far scendere l'errore rilevato sotto la soglia critica. Nelle implementazioni reali, la trasmissione di migliaia di impulsi garantisce che il tasso di errore converga verso il valore teorico del 25%, rendendo l'intercettazione matematicamente certa. Una volta rilevata un'anomalia, Alice e Bob interrompono la procedura, scartano la chiave compromessa e iniziano una nuova sessione su un canale differente.

Capitolo 2

Implementazione in Qiskit

Questa tesina ha come obiettivo quello di realizzare l'implementazione del protocollo BB84 in assenza e in presenza di un intercettatore, mostrando la relativa fidelity e il tasso di errore, necessari per stabilire se la comunicazione è avvenuta in modo sicuro e quindi se è possibile utilizzare la chiave ottenuta.

2.1 Strumenti Software: Qiskit e Aer

L'implementazione del protocollo BB84 è stata realizzata utilizzando **Qiskit**, un framework open-source sviluppato da IBM per la programmazione di computer quantistici a livello di circuiti e impulsi.

In particolare, il cuore dello sviluppo si basa su:

- **Qiskit SDK**: fornisce le classi fondamentali come `QuantumCircuit` per la costruzione logica dell'algoritmo e la gestione dei registri.
- **AerSimulator**: componente dedicato alla simulazione ad alte prestazioni. Per l'implementazione del protocollo è stato utilizzato l'AerSimulator in quanto permette di emulare un ambiente locale ideale e *noise-free*. Tale scelta è cruciale in fase di test, poiché consente di verificare la correttezza teorica del protocollo senza le interferenze tipiche dell'hardware quantistico attuale, garantendo che ogni errore rilevato sia riconducibile esclusivamente all'azione di un intercettatore.

2.2 Gate Quantistici e Operatori

Le manipolazioni degli stati quantistici avvengono tramite i gate quantistici, l'equivalente funzionale delle porte logiche nell'informatica classica. I gate fondamentali in questa implementazione sono:

- **Porta Pauli-X (X):** Equivale al gate NOT classico. Inverte lo stato da $|0\rangle$ a $|1\rangle$ e viceversa. In Qiskit è fondamentale perché i qubit vengono

inizializzati nello stato $|0\rangle$; per codificare il valore binario "1", è necessario applicare una porta X.

- **Porta di Hadamard (H):** È essenziale per il protocollo BB84 poiché permette di creare la sovrapposizione degli stati, consentendo il passaggio dalla base computazionale (Z) alla base diagonale (X):
  - Trasforma $|0\rangle$ in $|+\rangle$: $\frac{|0\rangle+|1\rangle}{\sqrt{2}}$
  - Trasforma $|1\rangle$ in $|-\rangle$: $\frac{|0\rangle-|1\rangle}{\sqrt{2}}$

Nota implementativa: Per inviare il bit 1 in base X ($|-\rangle$), Alice deve applicare prima il gate X (per ottenere $|1\rangle$) e poi il gate H. In fase di decodifica, Bob utilizza il gate H per "ruotare" i qubit ricevuti se sceglie di misurare nella base diagonale, riportandoli nella base rettilinea leggibile dal simulatore.

2.3 Analisi del codice

Di seguito viene analizzata l'architettura logica del software sviluppato, evidenziando le fasi cruciali del protocollo BB84 tradotte in istruzioni Qiskit.

1. Setup iniziale Vengono inizializzati i qubit necessari e definiti i parametri casuali per Alice. Utilizzando il generatore di numeri casuali di *NumPy*, Alice determina la stringa di bit da trasmettere e le basi di codifica.

```
rng = np.random.default_rng()
bit_num = 20  # Numero di qubit inviati
qc = QuantumCircuit(bit_num, bit_num)

# Alice genera bit e basi casuali
abits = rng.integers(0, 2, size=bit_num)
abase = rng.integers(0, 2, size=bit_num)
```

2. Preparazione degli stati Poiché i qubit in Qiskit vengono inizializzati di default nello stato $|0\rangle$, Alice deve orientarli correttamente. Se il bit scelto è 1, applica il gate **X**. Successivamente, se la base scelta è quella diagonale (X), applica il gate di **Hadamard** per creare la sovrapposizione.

```
for n in range(bit_num):
    if abits[n] == 1:
        qc.x(n)      # Porta lo stato da |0> a |1>
    if abase[n] == 1:
        qc.h(n)      # Trasforma |0> in |+> e |1> in |->
```

3. Esecuzione sul simulatore Il circuito viene inviato all'ambiente `AerSimulator`. In questa fase viene simulato il passaggio nel canale quantistico e l'interazione fisica dei qubit.

```
backend = AerSimulator()
job = backend.run(qc, shots=1, memory=True)
result = job.result()
```

4. Misurazione di Bob Bob sceglie casualmente le proprie basi. Se queste corrispondono a quelle di Alice, la misurazione proietterà il qubit sul valore corretto; in caso contrario, si otterrà un risultato casuale. I dati vengono estratti dalla memoria del job, invertendo la stringa per correggere l'ordinamento *little-endian* di Qiskit.

```
bbase = rng.integers(0, 2, size=bit_num)
# Estrazione delle misure dal simulatore
bob_measurements = [int(b) for b in result.get_memory()
    [0][::-1]]
```

5. Sifting e calcolo della chiave In questa fase classica, Alice e Bob confrontano le basi utilizzate. Solo i bit corrispondenti alla medesima base vengono mantenuti per formare la chiave segreta.

```
for i in range(bit_num):
    if abase[i] == bbase[i]: # Se le basi coincidono
        chiave_alice.append(int(abits[i]))
        chiave_bob.append(bob_measurements[i])
        if chiave_alice[-1] == chiave_bob[-1]:
            match_count += 1
```

6. Calcolo delle metriche di sicurezza Infine, vengono determinate la **Fidelity** e la **Loss** (QBER). Queste metriche sono i "sensori" del protocollo: una Fidelity inferiore al 100% in un ambiente simulato indica univocamente un'intercettazione o un errore di sistema.

```
if len(chiave_alice) > 0:
    fidelity = match_count / len(chiave_alice)
    loss = 1 - fidelity
```

NB: Per ulteriori approfondimenti vedere il codice `bb84.py` e `bb84_eve.py`

Capitolo 3

Analisi dei Risultati

3.1 Esempio d'esecuzione senza intercettazione

Nel grafico e nell'output successivi viene analizzata un'esecuzione del protocollo BB84 in uno scenario ideale, privo di interferenze esterne. In questa specifica istanza, il destinatario (Bob) ha selezionato casualmente la base corretta per 12 dei 20 qubit trasmessi da Alice.

La chiave grezza finale viene quindi generata estraendo esclusivamente i bit in cui vi è stata concordanza tra le basi di Alice e Bob (fase di *sifting*). In assenza di un intercettatore e di rumore hardware nel simulatore, la **Fidelity** della trasmissione raggiunge il 100%, con un tasso di errore (**QBER**) pari a zero. Questo conferma la correttezza teorica dell'implementazione: quando le basi coincidono, lo stato quantistico misurato da Bob corrisponde esattamente a quello preparato da Alice, garantendo l'integrità totale della chiave scambiata.

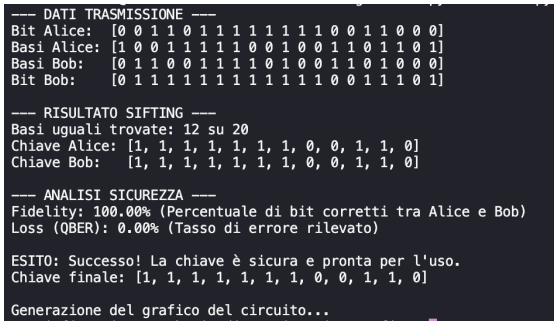
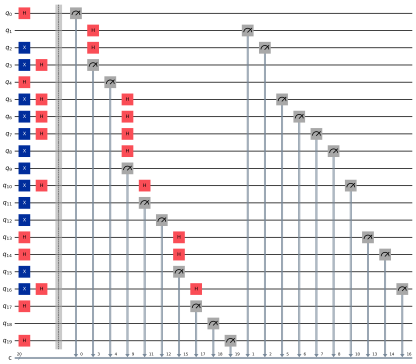


Figura 3.1: Esecuzione di BB84 senza intercettazione.

Rappresentazione grafica del circuito Il circuito rappresentato in figura illustra l'intera pipeline della comunicazione tra Alice e Bob. È possibile distinguere due fasi, evidenziate dalle barre grigie verticali:

- **Prima fase**: Alice deve codificare i bit scelti. I gate **X** (blu) e **H** (rossi) prima della barra verticale grigia indicano la codifica del bit e la scelta della base. Dove appare solo il gate H, Alice sta inviando un bit 0 in base diagonale; dove appaiono entrambi, invia un bit 1 in base diagonale.
- **Seconda fase**: Bob applica i propri gate H in modo casuale ed effettua la misura finale, rappresentata dal disegno del tachimetro. I risultati vengono registrati nel registro C visibile nella parte bassa dello schema.



3.2 Esempio d'esecuzione con intercettazione

3.2.1 Eve intercettata

In questa seconda simulazione viene analizzato l'impatto della presenza di un ascoltatore nella comunicazione (Eve) che attua un attacco di tipo **Intercept-Resend**. Come evidenziato dai dati raccolti, il tasso di errore (QBER) riscontrato è pari al 33,33%. Questo valore, superando abbondantemente la soglia critica di sicurezza dell'11%, fornisce ad Alice e Bob la prova statistica certa di un'intrusione nel canale quantistico.

```
--- DATI TRASMISSIONE ---
Bit trasmessi da Alice:  [0 1 1 0 1 1 0 1 0 1 1 0 0 0 0 0 0 0 1 1]
Basi scelte da Alice: [0 1 0 0 0 1 1 1 0 0 1 0 0 0 0 0 1 1 0 1]
Basi scelte Bob:   [1 1 1 0 0 0 0 0 1 1 1 0 0 1 0 0 0 0 0 0]
Bit letti da Bob:    [0 1 0 1 1 1 0 0 1 0 1 0 0 1 0 1 0 0 0 1]

--- ANALISI INTERCETTAZIONE ---
Basi scelte da Eve:   [0 0 1 1 1 1 1 1 1 0 1 0 0 1 0 1 0 0 1 0]
Bit letti da Eve: [0 0 0 1 1 1 0 1 1 1 1 0 0 1 0 1 0 0 0 1]
Chiave di Eve:    [0, 1, 1, 1, 0, 0, 0, 1, 0]

--- RISULTATO SIFTING ---
Basi uguali trovate: 9 su 20
Chiave di Alice: [1, 0, 1, 1, 0, 0, 0, 0, 1]
Chiave di Bob:   [1, 1, 1, 1, 0, 0, 0, 1, 0]

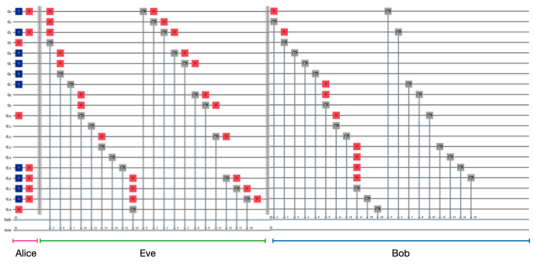
--- ANALISI SICUREZZA ---
Fidelity: 66.67% (Percentuale di bit corretti tra Alice e Bob)
Loss (QBER): 33.33% (Tasso di errore rilevato)

ATTENZIONE: Possibile presenza di Eve! Il tasso di errore supera l'11%.

Generazione del grafico del circuito...
```

Rappresentazione grafica del circuito Il circuito illustra l'intera pipeline della comunicazione mediata dall'intervento di Eve. Si possono distinguere tre fasi delimitate dalle barriere grigie verticali:

- **Prima Fase (Preparazione)**: Alice codifica i qubit secondo i bit e le basi scelti casualmente.
- **Seconda Fase (Intercettazione)**: Rappresenta l'operazione non passiva di Eve. Si nota l'applicazione del gate **H** (qualora scelga la base diagonale) seguita dalla misura intermedia (il simbolo del tachimetro). Questa azione provoca il **collasso della funzione d'onda**: se la base di Eve è errata, lo stato viene alterato permanentemente. Eve applica nuovamente il gate **H** prima di inoltrare il qubit, ma l'informazione originale è ormai compromessa.
- **Terza Fase (Ricezione)**: Bob applica i propri gate **H** e procede alla misura finale. I risultati vengono registrati nei due registri classici distinti (`bob` ed `eve`), permettendo di confrontare i dati estratti dall'intercettatore con quelli ricevuti dal destinatario e calcolare il tasso di errore introdotto.



3.2.2 Eve non intercettata

Come evidenziato dai dati ottenuti nella simulazione con la presenza di Eve, il tasso di errore (QBER) può talvolta assestarsi su valori inferiori alla soglia critica dell'11%, nonostante l'intercettazione attiva. Tale fenomeno non inficia la

validità del protocollo, ma è una diretta conseguenza della **dimensione ridotta del campione** utilizzato (20 bit) a scopo didattico.

In statistica, campioni di piccola entità sono soggetti a forti fluttuazioni: in questo scenario, è probabile che l'intercettatore abbia scelto casualmente le basi corrette con una frequenza superiore alla media teorica del 50%, o che gli errori introdotti siano ricaduti su bit successivamente scartati durante la fase di *sifting*. Nelle implementazioni reali del protocollo BB84, dove vengono trasmessi migliaia di qubit, la probabilità che il QBER rimanga sotto la soglia di sicurezza in presenza di un attacco diventa matematicamente trascurabile.

3.3 Analisi Statistica e Convergenza

Per approfondire la validità dei risultati ottenuti, è stata effettuata una simulazione iterativa su un campione più ampio utilizzando un *Jupyter Notebook*. L'obiettivo di questa analisi è dimostrare come la precisione nel rilevamento di Eve sia strettamente correlata alla lunghezza della chiave scambiata.

3.3.1 Dati Sperimentali

Il protocollo è stato eseguito per 10 iterazioni indipendenti per tre diverse lunghezze di chiave ($N = 20, 100, 1000$). I risultati sono riassunti nella seguente tabella:

Taglia Chiave	**QBER Medio**	**Deviazione Standard**
20 bit	21.40%	0.0838
100 bit	22.08%	0.0776
1000 bit	25.14%	0.0169

Tabella 3.1: Analisi della convergenza statistica del QBER.

3.3.2 Interpretazione dei Risultati

Il grafico in figura non si limita a mostrare il valore medio del QBER, ma offre una visione della stabilità statistica del protocollo attraverso la struttura dei *Box Plot*:

- **Lunghezza dei Box e dei Baffi:** La dimensione verticale del "box" (l'intervallo interquartile) e l'estensione dei "baffi" indicano la variabilità dei risultati. Per $N = 20$, i baffi sono molto estesi, evidenziando che il tasso di errore è fortemente influenzato dalla casualità delle basi scelte in un campione ridotto. Al contrario, per $N = 1000$, il box appare estremamente compresso, indicando che i risultati di diverse simulazioni sono quasi identici e altamente prevedibili.
- **Allineamento della Mediana:** La linea orizzontale all'interno di ogni box rappresenta la mediana dei risultati. Si osserva che per $N = 1000$

la mediana si posiziona quasi perfettamente sulla linea rossa del **25%**. Questo allineamento conferma la correttezza della simulazione: teoricamente, Eve induce un errore del 50% sui bit che intercetta con la base sbagliata (che sono la metà del totale), portando l'errore complessivo atteso esattamente al 25%.

- **Affidabilità della soglia di sicurezza:** Osservando il baffo inferiore del test a 20 bit, si nota come questo si avvicini alla soglia dell'11% (linea verde). Ciò implica un rischio reale di *falsi negativi* su campioni piccoli. L'analisi dimostra che solo aumentando il numero di qubit (passando a $N = 1000$) la distanza tra l'errore minimo misurato e la soglia di sicurezza diventa tale da garantire l'identificazione di Eve in ogni scenario possibile.

In sintesi, il passaggio da un box plot "lungo e instabile" a uno "corto e centrato" è la prova visiva della transizione da un regime di forte incertezza statistica a uno di precisione deterministica, tipico delle comunicazioni quantistiche su larga scala.

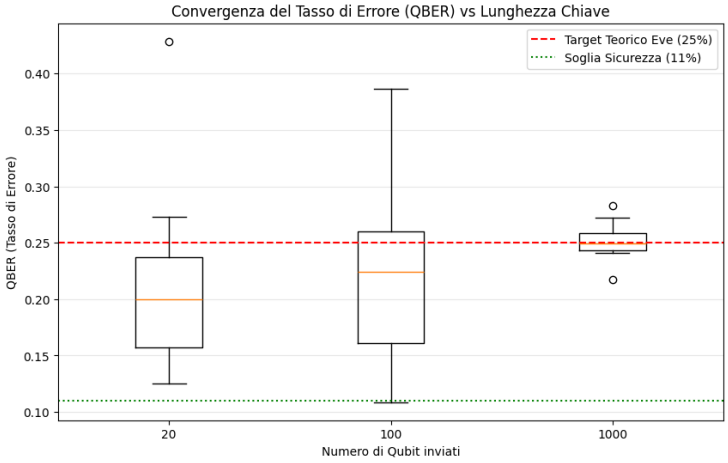


Figura 3.2: Boxplot della convergenza del tasso di errore. Si nota come per $N = 1000$ la varianza sia minima e il valore centrato sul target teorico.

Conclusioni

Il presente lavoro ha permesso di esplorare le potenzialità della crittografia quantistica attraverso la simulazione del protocollo BB84. L'implementazione in ambiente Qiskit e la successiva analisi statistica hanno confermato che la sicurezza di questo sistema non risiede nella complessità computazionale, come nei crittosistemi classici, ma nelle leggi fondamentali della fisica quantistica.

Dalle simulazioni e dai test di convergenza effettuati sono emersi tre punti chiave:

- **Inviolabilità teorica**: In assenza di disturbi, il protocollo garantisce la generazione di una chiave segreta perfetta (Fidelity 100%), confermando l'efficacia della codifica quantistica e la corretta implementazione dei gate di Hadamard e Pauli-X per la gestione delle basi.
- **Determinismo della rilevazione**: L'analisi su larga scala (fino a 1000 qubit) ha dimostrato che l'intercettazione da parte di Eve non è un evento incerto, ma un fenomeno che converge stabilmente verso un tasso di errore (QBER) del 25%. Questo convalida il modello teorico dell'attacco **Intercept-Resend** e la natura non passiva della misura quantistica.
- **Validazione della soglia di sicurezza**: Lo studio della deviazione standard ha evidenziato come la "soglia dell'11%" diventi una barriera di sicurezza invalicabile all'aumentare di N. Mentre su campioni ridotti esiste un rischio statistico di falsi negativi, la simulazione iterativa ha provato che su scale reali la distinzione tra rumore hardware e intercettazione attiva è netta e matematicamente garantita.

In conclusione, il protocollo BB84 rappresenta un pilastro della *Quantum Key Distribution*. Nonostante le sfide tecnologiche legate alla sensibilità dell'hardware reale, questa implementazione dimostra che la QKD è la soluzione più promettente per garantire la *Perfect Forward Secrecy*. In un futuro dominato dal calcolo quantistico, protocolli come il BB84 renderanno obsoleti i metodi di intercettazione passiva tramite l'attacco **Intercept-Resend**, spostando il paradigma della sicurezza dalla complessità degli algoritmi alla certezza delle leggi fisiche.

Riferimenti Bibliografici

[1] S. Pirandola et al., “Advances in Quantum Cryptography”, IEEE, 2020. https://ieeexplore.ieee.org/abstract/document/9990073